gems/activerecord-2.3.2/lib/active_record/attribute_methods.rb

```
module ActiveRecord
  module AttributeMethods
    def method_missing(method_id, *args, &block)
      # ...

      if self.class.primary_key.to_s == method_name
        id
      elsif md = self.class.match_attribute_method?(method_name)
        attribute_name, method_type = md.pre_match, md.to_s
        if @attributes.include?(attribute_name)
          __send__("attribute#{method_type}", attribute_name, *args, &block)
        else
          super
        end
      elsif @attributes.include?(method_name)
        read_attribute(method_name)
      else
        super
      end
    end

    private
      # Handle *? for method_missing.
      def attribute?(attribute_name)
        query_attribute(attribute_name)
      end

      # Handle *= for method_missing.
      def attribute=(attribute_name, value)
        write_attribute(attribute_name, value)
      end
```

Look at the code in method_missing above. If you're accessing the object's identifier, then it returns its value. If you're calling an attribute accessor, then it calls the accessor with either a *Dynamic Dispatch* (49) (for write or query accessors) or a direct call to read_attribute (for read accessors). Otherwise, method_missing sends the call up the chain of ancestors with super.

I don't want to waste your time with unnecessary details, so I only showed you part of the code for attribute methods in Rails 2. What you've seen, however, shows that both the feature and its code became more complicated in the second major version of Rails. Let's see how this trend continued in the following versions.